PLATEFORME CONSEIL EN IMAGE ARIANE DAGO

PRÉSENTATION APPLICATION & STACK TECHNIQUE

Document de référence décrivant l'application web (parcours client, e-commerce, back-office) et l'ensemble des technologies utilisées pour le développement, le déploiement et l'exploitation.

```
Projet : conseil-image-ariane (v0.1.x)
Dépôt : Serge173/ariane_app
Runtime : Node.js 24 · npm 10+
Région Vercel : cdg1 (Paris)
Développement : KEAYENI SERGE PACOME
```

KEAYENI SERGE PACOME

Dossier technique · 2026 · Développement KEAYENI SERGE PACOME



1. VUE D'ENSEMBLE DE L'APPLICATION

Application **full-stack** unique : site vitrine premium, parcours de réservation et de paiement, boutique en ligne, espace client, blog, newsletter et **back-office administrateur** complet. Conçue **mobile-first**, avec une identité visuelle sobre (inspiration luxe / Polène).

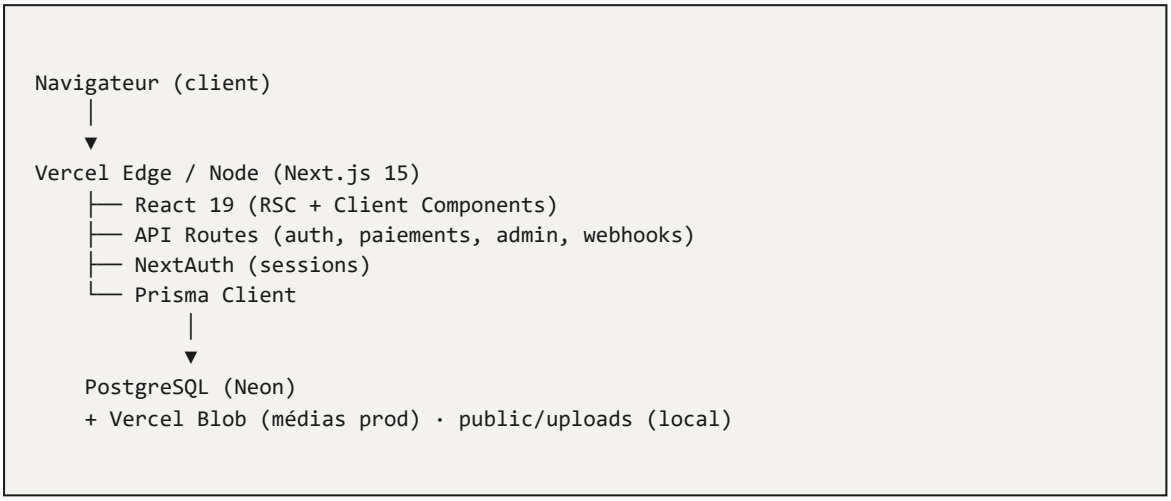
> **Parcours métier :** Prospect → Orientation / découverte → Prestation ou produit → Rendez-vous → Paiement (CinetPay) → Coaching & questionnaires → Suivi (Mon espace) → Fidélisation (newsletter, blog).

Modules fonctionnels principaux

MODULE	RÔLE
Site public	Accueil, offres, orientation, réservation, panier, checkout, blog, FAQ, contact, WhatsApp
Parcours découverte	Formulaire configurable (admin) + créneaux RDV, stocké en base (ContactRequest + metadata)
E-commerce	Produits, variantes (taille/couleur), stock, codes promo, commandes, expédition (fulfillment)
Mon espace client	Profil, commandes, RDV, documents, questionnaires
Admin	Contenu, offres, boutique, commandes, RDV, blog, newsletter, paiements, utilisateurs, analytics
Newsletter	Inscriptions, modèle d'email éditable, envoi manuel / auto à la publication blog, webhook externe

2. ARCHITECTURE LOGICIELLE

Next.js 15 avec **App Router** (`src/app/`) : pages React côté serveur et client, **Route Handlers** pour les API REST (`src/app/api/`). Pas de backend séparé : logique métier dans `src/lib/`, accès données via **Prisma**.



Patterns utilisés

- **Server Components** par défaut ; composants interactifs en `"use client"`
- **Zustand** pour l'état panier côté navigateur
- **React Hook Form + Zod** pour formulaires et validation
- **Settings JSON** en base (`SiteContent`) pour contenus éditables (hero, parcours découverte, newsletter)
- **Webhooks** : CinetPay (paiement), publication newsletter (Zapier/Make possible)

3. STACK FRONTEND

TECHNOLOGIE	VERSION / USAGE
React	19 — interface utilisateur
Next.js	15 — routing, SSR/SSG, images optimisées, build production
TypeScript	5.7 — typage statique sur tout le projet
Tailwind CSS	3.4 + PostCSS + Autoprefixer — design system utilitaire
Radix UI	Accordion, Dialog, Select, Tabs, Toast, etc. — accessibilité
class-variance-authority, clsx, tailwind-merge	Variants de composants UI réutilisables
Framer Motion	Animations (accueil, transitions)
Lucide React	Icônes
Recharts	Graphiques tableau de bord admin
date-fns	Dates (RDV, disponibilités)
next/image	Images Unsplash + Vercel Blob (remotePatterns)

4. STACK BACKEND & DONNÉES

TECHNOLOGIE	USAGE
PostgreSQL	Base relationnelle (hébergement recommandé : Neon)
Prisma ORM	Schéma, migrations / db push, client typé, seed (`tsx prisma/seed.ts`)
Modèles métier	User, Order, Product, ProductVariant, Payment, Appointment, BlogPost, Newsletter*, ContactRequest, …
bcryptjs	Hash des mots de passe
sharp	Traitement images (pipeline Next.js)

5. AUTHENTIFICATION & RÔLES

COMPOSANT	DÉTAIL
NextAuth.js	v4 — sessions, pages de connexion, protection routes admin
@auth/prisma-adapter	Persistance Account / Session / User en PostgreSQL
Rôles (`UserRole`)	CLIENT, ADMIN, SUPER_ADMIN, MANAGER_SHOP, MANAGER_ORDERS, ACCOUNTING

6. PAIEMENTS

Intégration **CinetPay** (Côte d'Ivoire et zone Mobile Money) : Orange, MTN, Moov, Wave, carte bancaire — canaux configurables en admin (`PaymentMethodConfig`).

- Création de transaction côté serveur (`src/lib/payments/cinetpay.ts`)
- Webhook de notification : `/api/payments/webhook`
- Statuts commande / paiement synchronisés en base (enums Prisma)

7. EMAILS & NEWSLETTER

- **Resend** — envoi transactionnel et campagnes (`src/lib/email.ts`)
- Modèle newsletter éditable en admin, variables dynamiques (contenu publication, liens réseaux)
- Envoi automatique à la publication d'un article blog (si activé)
- Webhook `/api/webhooks/newsletter-publication` pour automatisation externe

8. HÉBERGEMENT, INFRA & SERVICES CLOUD

SERVICE	RÔLE
Vercel	Hébergement Next.js, CI/CD Git, preview deployments, région cdg1
Neon (PostgreSQL)	Base de données serverless, `DATABASE_URL`
Vercel Blob	Stockage fichiers uploadés en production (`@vercel/blob`)
@vercel/analytics	Métriques audience (optionnel)
Google Analytics	Via `NEXT_PUBLIC_GA_ID` (composant AnalyticsScripts)

Build & déploiement

- Script `vercel-build` : `prisma generate` → `prisma db push` → prebuild → `next build`
- Configuration : `vercel.json`, `next.config.ts` (URL publique, images, redirects admin)
- Guide détaillé : `DEPLOY_VERCEL.md` à la racine du dépôt

9. OUTILS DE DÉVELOPPEMENT

ESLint 9 | eslint-config-next | tsx | Prisma Studio | Git / GitHub

Développement local : `npm run dev` (port 3001), `npm run db:push`, `npm run db:seed`, variables dans `.env` (voir `.env.example`).

10. STRUCTURE DU CODE SOURCE

```
src/
├── app/                 # Pages & API (App Router)
│   ├── admin/           # Back-office
│   ├── api/             # REST (payments, admin, public, webhooks)
│   ├── boutique/, offres/, reservation/, mon-espace/, blog/, …
├── components/          # UI (home, shop, admin, layout, …)
├── lib/                 # Métier (auth, prisma, payments, newsletter, store, …)
└── types/
prisma/
├── schema.prisma        # Modèle de données complet
└── seed.ts              # Données initiales (admin, catalogue)
scripts/                 # Prebuild Vercel, utilitaires
```

11. VARIABLES D'ENVIRONNEMENT (PRINCIPALES)

VARIABLE	USAGE
DATABASE_URL	Connexion PostgreSQL (Neon)
NEXTAUTH_URL / NEXTAUTH_SECRET	URL publique & secret sessions NextAuth
NEXT_PUBLIC_APP_URL	Liens absolus, emails, webhooks
CINETPAY_*	API Key, Site ID, URL notification
RESEND_API_KEY	Envoi d'emails
BLOB_READ_WRITE_TOKEN	Upload médias Vercel Blob
NEXT_PUBLIC_WHATSAPP_NUMBER	Bouton contact WhatsApp
NEXT_PUBLIC_GA_ID	Google Analytics (optionnel)

12. SYNTHÈSE STACK (LISTE RAPIDE)

Next.js 15 | React 19 | TypeScript | Tailwind CSS 3 | Radix UI | Framer Motion | Prisma 6 | PostgreSQL | NextAuth.js | Zod | React Hook Form | Zustand | CinetPay | Resend | Vercel | Neon | Vercel Blob | Recharts | Lucide | date-fns | bcryptjs | sharp | ESLint

13. PÉRIMÈTRE BACK-OFFICE (ADMIN)

- Tableau de bord & statistiques (Recharts, événements analytics en base)
- Gestion contenu site, hero, FAQ, parcours découverte (questions / textes)
- Offres, disponibilités, créneaux bloqués, demandes RDV (fiche, édition, impression)
- Boutique : produits, variantes, stock, marques, catégories, codes promo
- Commandes : statuts, fulfillment, paiements
- Blog (couverture, liens publication réseaux), newsletter (abonnés, envois, réglages)
- Méthodes de paiement, utilisateurs & rôles, guide intégré admin

> **Client :** Ariane DAGO — Conseil en image · Plateforme développée sur mesure pour l'accompagnement, la vente et la relation client digitale.
>
> **Développement :** KEAYENI SERGE PACOME — conception, développement et déploiement de l'application.

Fin du dossier technique · conseil-image-ariane · 2026
KEAYENI SERGE PACOME